

Q1. Explain the difference between Gradient Descent, Stochastic Gradient Descent (SGD), and Mini-batch Gradient Descent. Also compare them in terms of speed, stability, and practical usage.

The core difference lies in how much data is used to compute the gradient before updating the weights:

- **Gradient Descent (Batch GD):** Uses the entire dataset to perform a single weight update.
- **Stochastic Gradient Descent (SGD):** Uses a single random data sample to update weights for every iteration.
- **Mini-batch Gradient Descent:** Divides the dataset into small subsets (e.g., 32, 64, or 128 samples) and updates weights per batch.

Comparison

- **Speed:** SGD and Mini-batch are fast per iteration because they process less data at once; Batch GD is very slow on large datasets.
- **Stability:** Batch GD is stable and converges smoothly; SGD is noisy with heavy updates; Mini-batch balances both by reducing noise while maintaining faster iteration.
- **Practical Usage:** Mini-batch GD is the industry standard for training deep learning models because it leverages GPU parallel processing efficiently.

Q2. Why does Momentum help reduce oscillation and speed up learning? Explain using the idea of zig-zag movement and velocity.

Standard gradient descent often suffers from a **zig-zag movement** when the loss surface is steep in one direction and shallow in another (like a long valley). It bounces back and forth across the steep walls rather than moving straight down towards the minimum.

Momentum solves this by borrowing the physics concept of **velocity**:

1. It maintains a running average of past gradients.
2. In directions where gradients continually change sign (oscillating back and forth), the opposing values cancel each other out, dampening unwanted horizontal jumps.
3. In directions where gradients consistently point the same way, momentum accumulates speed, causing the update to accelerate straight down the slope toward the minimum.

Q3. What is the main problem of AdaGrad? Explain how it affects the learning rate during training.

The main problem with **AdaGrad** is its **continually decaying learning rate**.

AdaGrad accumulates the squares of all past gradients in the denominator to adapt the learning rate. Because squared values are always positive, this denominator grows monotonically with every step. As a result, the effective learning rate monotonically decreases and eventually becomes infinitely small, causing the model to stop learning completely long before reaching the optimal solution.

Q4. How does RMSProp solve the problem of AdaGrad? Explain the difference between full accumulation and moving average.

RMSProp solves AdaGrad's learning rate decay by replacing **full accumulation** with an **exponentially weighted moving average**.

- **Full Accumulation (AdaGrad):** Sums up *every* past squared gradient from the very beginning of training, causing the denominator to grow continuously without bound.
- **Exponential Moving Average (RMSProp):** Keeps track of only recent squared gradients by applying a discount factor (β). Older gradients gradually fade away, preventing the denominator from growing indefinitely and allowing the learning rate to remain effective throughout training.

Q5. Why is Adam considered a combination of Momentum and RMSProp? Explain in simple terms.

Adam combines the best features of both algorithms into a single optimizer:

- **Momentum's Contribution:** Uses the exponentially weighted average of past gradients (first moment) to accelerate along consistent directions and smooth out oscillations.
- **RMSProp's Contribution:** Uses the exponentially weighted average of past *squared* gradients (second moment) to adaptively scale the learning rate for each individual parameter based on recent gradient history.

In short, Momentum directs **which way** to move smoothly, and RMSProp determines **how far** to step for each parameter.

Q6. Numerical Calculation

To calculate the updated weight using SGD, we use the standard gradient descent update rule:

Updated Weight $\Rightarrow w_{\text{new}} = w - \text{learning rate} * dw$

Given:

- $w = 5$
- $dw = 2$
- learning rate = 0.1

calculation:

$$w_{\text{new}} = 5 - (0.1 * 2)$$

$$w_{\text{new}} = 5 - 0.2$$

$$w_{\text{new}} = 4.8$$

The updated weight is **4.8**.

Q7. Which optimizer would you choose for:

a) Sparse data (like NLP tasks)

- **Choice: AdaGrad or Adam**
- **Reason:** Sparse datasets (like text representations) contain features that appear infrequently. AdaGrad and Adam perform larger weight updates for infrequent features and smaller updates for frequent ones, ensuring rare but important words/tokens still get trained effectively.

b) Deep neural networks

- **Choice: Adam (or AdamW / SGD with Momentum)**
- **Reason:** Deep networks suffer from complex, high-dimensional loss landscapes, noisy gradients, and saddle points. Adam combines adaptive learning rates for fast convergence with momentum to traverse narrow valleys and navigate noisy landscapes effectively.

Q8. If your model is learning very slowly, which method would you try: Momentum, RMSProp, or Adam? Explain your answer.

Choice: **Adam** , because :

Adam is the best first choice because it tackles slow training from two fronts simultaneously:

1. **Velocity (Momentum):** If the gradients keep pointing in the same direction, Adam builds up speed to push through flat regions (plateaus) quickly.
2. **Adaptive Learning Rates (RMSProp):** If certain parameters have very small gradients, Adam scales up their step sizes dynamically so they don't lag behind.

While standard Momentum or RMSProp address only one of these bottlenecks, Adam targets both, giving you the fastest convergence with minimal hyperparameter tuning.